

Claude Code Quick Reference

AIA Copilot — Claude Code Training (August 2026 edition)

Claude Code is an agentic coding tool that runs in your terminal. It reads your codebase, plans changes, writes code, runs commands, and fixes its own errors. You provide intent; Claude provides implementation.

Installation (Native — no Node/npm)

The npm install path was deprecated January 2026. Use the native installer:

OS	Command
macOS / Linux	Run the claude.ai install script (<code>curl -fsSL https://claude.ai/install.sh</code> piped to <code>bash</code>) or <code>brew install claude-code</code>
Windows	<code>winget install Anthropic.ClaudeCode</code> (native — no WSL)
Linux pkg	apt / dnf / apk packages available

Verify: `claude --version` → v2.1.219+ for all course features. Confirm the approved identity, `/usage`, and `/model`; organization policy and provider support determine live access.

Models (August 2026)

Alias	Role
sonnet / Sonnet 5	Daily implementation and debugging; common subscription default
opus / Opus 5	Complex reasoning and architecture; Claude Code v2.1.219+
fable / Fable 5	Hardest, longest autonomous work when available
haiku / latest Haiku	Fast, low-cost mechanical work

Use `/model` to verify account availability and switch deliberately. Administrators and providers can restrict or remap available models.

Essential Commands

Command	What it does
<code>claude</code>	Start in current directory
<code>claude -p "..."</code>	Authenticated one-shot using the existing Claude Code login. Add <code>--tools</code> to restrict, matching <code>--allowedTools</code> to pre-approve, <code>--permission-mode dontAsk</code> , and <code>--max-turns</code>
<code>/clear</code>	Reset conversation context; saved skill and hook edits normally live-detect
<code>/compact</code>	Compress history, keep working
<code>/context</code>	Show what's in the window and what it costs
<code>/usage</code>	Plan, session + weekly usage bars, reset timers, and usage attributed by skill / subagent / plugin / MCP server
<code>/model</code>	Switch models
<code>/doctor</code>	Installation and configuration health checks, including skill-listing budget contributors
<code>/btw <q></code>	Ephemeral side question — no context pollution
<code>/memory</code>	View CLAUDE.md + Claude's auto-memory notes
<code>/permissions</code>	Interactive permission rules menu
<code>/</code> <code>skills</code> <code>/</code> <code>agents</code> <code>/plugin</code>	Inventory skills / subagents / plugins; use <code>/doctor</code> for setup and listing-budget contributors
<code>/keybindings</code>	Customize keys → <code>~/.claude/keybindings.json</code>
<code>/init</code>	Bootstrap a CLAUDE.md from a codebase scan
<code>/rewind</code>	Open the rewind menu (same as double-Esc)

Keyboard Essentials

The VS Code panel is our primary surface. Where the two differ, use the panel row.

Keys	Action	Surface
Shift+Tab	Cycle permission modes: Manual -> AcceptEdits -> Plan -> Auto	Both
Alt+K	Insert an @-mention of the current file and your selected line range (<code>@app.ts#5-10</code>)	Panel
Ctrl+Esc	Toggle focus between the editor and Claude's prompt box	Panel
Shift+Enter	New line without sending	Both
Ctrl+Alt+F	Focus view - collapse tool calls and results behind one row per turn	Panel
Ctrl+Shift+Esc	Open a new conversation as an editor tab	Panel
Ctrl+Shift+T	Reopen the most recently closed Claude session tab	Panel
Hover a message -> rewind arrow	Rewind menu: fork conversation / rewind code / both	Panel
Esc Esc (double-tap)	The same rewind menu	Terminal
Esc (single)	Stop - cancels the running tool call immediately	Both
<i>Type + Enter while it works</i>	Steer - sends without stopping; Claude reads it after the current action and adjusts	Both
<code>@file</code>	Load a file, folder, or URL into context	Both
<code>!command</code>	Run a shell command directly	Terminal only - not in the panel

Steer vs stop. These are different tools. `Esc` kills the action in flight. Typing a correction and hitting Enter leaves the action running and course-corrects the next decision. Reach for typing first - it is the cheapest rung on the recovery ladder and it costs you nothing you already had.

VS Code Panel - Things That Save You Time

- **Your selection is already in context.** Highlight code and Claude sees it; the prompt-box footer shows the line count. Click that indicator to toggle it off - an eye-slash means the selection is hidden. `Alt+K` promotes the selection into an explicit `@file#5-10` mention you can see in your prompt.
- `@terminal:name` pulls a terminal's output into your prompt. Stack traces without copy-paste.
- **Plan mode opens a real Markdown document.** You can add inline comments in it before approving. Reviewing a plan is cheaper than reviewing a diff.
- **Focus view** (`Ctrl+Alt+F`) hides tool noise behind one expandable row per turn. The best setting for a shared screen or a projector. Applies to every open session immediately and persists.
- **Session history** (button at the top of the panel): search by keyword or browse by day. Sessions get AI-generated titles; hover to rename or remove.
- **Multiple conversations:** *Open in New Tab / New Window* from the Command Palette. On a tab's spark icon, a **blue dot** means a permission request is waiting; **orange** means Claude finished while hidden.
- `/usage` opens Account & usage - plan limits, reset timers, and attribution showing which skills, subagents, plugins, and MCP servers consumed your quota.
- **Extension settings** (`Ctrl+,` -> Extensions -> Claude Code): `initialPermissionMode` , `useTerminal` , `preferredLocation` , `autosave` , `respectGitIgnore` .

The Four Permission Modes

Mode	Behavior
Default	Asks before edits and risky commands
AcceptEdits	Auto-approves file edits
Plan	Read-only — plans, never changes
Auto	Autonomous within your permission rules; ships with two-layer safety (injection probe + transcript classifier), rm -rf guards

Permission Rules (settings.json)

Three arrays: `deny`, `ask`, `allow` — `deny` is evaluated first, then ask, then allow. A deny match wins even when an allow rule also matches. Configure via `/permissions` (menu writes the JSON for you). Enterprise: `managed-settings.json` is locked; `managed-settings.d/*.json` merge alphabetically.

Checkpoints & Rewind

Claude Code checkpoints continuously. Double-Esc → pick a checkpoint → restore: - **conversation** — bad instructions / derailed session - **code** — bad file changes - **both** — full undo

Rewind beats arguing with contaminated context for covered conversation and direct edits. **Do not assume it restores Bash side effects, subagent/background edits, or external changes.** Commit before fan-out; Git is the durable recovery layer. For damage that predates checkpoints, describe what is wrong, point at `@reference` files, and state what done looks like.

The `.claude` Directory - Where Things Live

Project scope lives in your repo under `.claude/` (or the repo root for `CLAUDE.md` and `.mcp.json`). Global scope lives in `~/.claude/` and applies to every project.

You want to	Edit	Commit it?
Give Claude project context and conventions	<code>CLAUDE.md</code>	Yes
Keep private preferences out of git	<code>CLAUDE.local.md</code> , <code>.claude/settings.local.json</code>	No - gitignore
Allow or block tool calls	<code>.claude/settings.json</code> -> <code>permissions</code>	Yes
Run a script before/after a tool call	<code>.claude/settings.json</code> -> <code>hooks</code>	Yes
Topic-scoped instructions, optionally path-gated	<code>.claude/rules/*.md</code>	Yes
A prompt you invoke with <code>/name</code>	<code>.claude/skills/<name>/SKILL.md</code>	Yes
A subagent with its own tools	<code>.claude/agents/*.md</code>	Yes
Team-shared MCP servers	<code>.mcp.json</code> (repo root)	Yes
Change how Claude formats responses	<code>.claude/output-styles/*.md</code>	Yes

Data Claude Code writes as you work (all under `~/.claude/` , all plaintext):

Path	Contents	Cleanup
<code>projects/<project>/<session>.jsonl</code>	Full transcript: every message, tool call, tool result	After <code>cleanupPeriodDays</code> (default 30)
<code>file-history/<session>/</code>	Pre-edit snapshots that power checkpoint restore	Last 100 checkpoints
<code>history.jsonl</code>	Every prompt you have ever typed	Never - until you delete it
<code>plans/</code> , <code>debug/</code> , <code>paste-cache/</code> , <code>image-cache/</code>	Plan files, logs, pasted content, images	After <code>cleanupPeriodDays</code>

Transcripts are not encrypted. If a tool reads a `.env` or a command prints a token, that value is on disk. Mitigate with a `Read` deny rule for credential files, a lower `cleanupPeriodDays`, or `CLAUDE_CODE_SKIP_PROMPT_HISTORY=1`.

Wipe one project's local state: `claude project purge ~/path/to/repo` (add `--dry-run` to preview). Do not delete `~/.claude.json`, `~/.claude/settings.json`, or `~/.claude/plugins/` - those hold your auth, preferences, and installed plugins.

CLAUDE.md Scopes

Scope	File	Shared?
Personal (global)	<code>~/.claude/CLAUDE.md</code>	No
Personal (project)	<code>CLAUDE.local.md</code>	No (gitignore it)
Team	<code>CLAUDE.md</code> in repo	Yes
Subdirectory	<code>subfolder/CLAUDE.md</code>	Loaded in that folder

Keep under ~200 lines. **Auto memory:** Claude also keeps its own notes (build commands, insights) loaded alongside CLAUDE.md — you curate the rules, Claude curates its learnings.

CLI for MCP & Plugins

Command	What it does
<code>claude mcp add --transport http/stdio</code> <code><name> <target></code>	Register an MCP server
<code>claude mcp login <name></code> / <code>claude mcp</code> <code>logout <name></code>	Shell auth for MCP servers (week 28)
<code>claude mcp list</code> / <code>remove</code>	Inventory / prune
<code>claude plugin init <name> --with skills</code> <code>hooks</code>	Create a skills-directory plugin under <code>~/.claude/skills/<name>/</code>
<code>/plugin marketplace add <path-or-repo></code>	Add a marketplace
<code>/plugin install <name>@<marketplace></code>	Install a plugin; follow the summary if <code>/reload-</code> <code>plugins</code> is required
<code>claude plugin list</code> / <code>details</code>	Inspect installed state and contributed components
<code>/plugin disable</code> / <code>uninstall</code>	Deactivate or remove at an intentional scope

Context Discipline

Compaction controls

Command	What it does
<code>/context</code>	Live breakdown of what is using your window, by category
<code>/compact</code>	Summarize now and keep going
<code>/compact focus on the auth bug</code>	Summarize but keep what <i>you</i> choose, not what it guesses
<code>/autocompact 500k</code>	Set how full the window gets before auto-compaction fires (100K-1M)
<code>/clear</code>	Hard reset between unrelated tasks

What survives a compaction

Loaded how	After compaction
Project-root <code>CLAUDE.md</code> , unscoped rules	Re-injected from disk
Auto memory (<code>MEMORY.md</code>)	Re-injected from disk
System prompt, output style	Unchanged - never in message history
Rules with <code>paths:</code> frontmatter	Lost until a matching file is read again
Nested <code>CLAUDE.md</code> in subfolders	Lost until a file in that folder is read again
Invoked skill bodies	Re-injected; capped 5K/skill, 25K total, oldest dropped first

*If a rule must survive compaction, put it in the **project-root** `CLAUDE.md` - not a nested file, not behind a `paths:` filter.*

- 1M-token window on Sonnet 5 — but treat it as a budget; compact around 50%.
- `@` mentions load exactly what you need; vague scoping can cost 50K tokens.
- MCP ToolSearch defers schemas by default. Use `/context` to inspect actual loading, review `alwaysLoad` exceptions, and disable unused servers.

- Same task = keep context. New task = `/clear` .

Troubleshooting

Problem	Fix
Claude stops mid-task	<code>exit</code> → <code>claude</code> → "Continue where you left off"
Off the rails	Double-Esc, rewind to the last good checkpoint
Convention not followed	CLAUDE.md in root? <code>/clear</code> to reload
Skill won't fire	Pushier description; check <code>/skills</code> ; <code>/clear</code>
Hook won't fire	Exact-case matcher (<code>Write</code> not <code>write</code>); <code>/clear</code> ; <code>/doctor</code>
Slow/pricey	<code>/model</code> down a tier; <code>/compact</code> ; check <code>/usage</code>
Anything weird	<code>/doctor</code> , press <code>f</code>

Access Paths (August 2026)

Access path	Verify live
Pro or Max subscription	Correct identity, <code>/usage</code> , and <code>/model</code>
Team or Enterprise seat	Organization policy, seat allowance, approved models
Anthropic Console / API	Credential path and token-billing owner
Cloud provider	Provider authentication, model support, and budgets

Classroom rule: prove access in the live CLI before class; do not infer it from a remembered price table.

Resources

- Docs: <https://code.claude.com/docs>
- Changelog: <https://code.claude.com/docs/en/whats-new>
- Azure Pipelines: use an authorized self-hosted agent where Claude Code is already installed and authenticated; run `claude -p` with restrictive `--tools`, matching pre-approval, `dontAsk`, and bounds
- Optional GitHub comparison: `anthropics/claude-code-action@v1`

© 2026 AIA Copilot — Claude Code Training